

TD/TP 2 — Du C au C++ moderne

D'une API C à une première interface C++

MAM4 — Programmation C++ Durée : 2 heures

Objectif. Faire évoluer un tableau dynamique d'entiers déjà connu en C. À chaque étape, partir d'une question concrète : faut-il lire ou modifier ? Que faire lorsque le tableau est plein ? Qui libère la mémoire ?

Les sources sont fournies dans l'archive `code-td-seance-02.zip`. Après extraction, `make` compile la version C et la première version C++. Conserver ensuite deux versions : une avec des fonctions libres, puis une avec des fonctions membres.

Organisation. 0–20 min : étape 0 ; 20–50 min : étape 1 ; 50–75 min : étapes 2–3 ; 75–105 min : étapes 4–5 ; 105–120 min : étape 6, bilan et dépôt. L'étape 6 est indispensable, même si certains tests doivent être abrégés.

Étape 0 — Comprendre le tableau C avant de le transformer

Lire `int_array_c.h`, `int_array_c.c` et `main_c.c`. Identifier le rôle des trois champs :

- `capacity` : nombre de places disponibles ;
- `size` : nombre d'éléments réellement utilisés ;
- `data` : adresse de la zone qui contient les entiers.

Pour que le tableau reste correct, les règles suivantes doivent rester vraies :

$0 \leq \text{size} \leq \text{capacity}$ $\text{capacity} == 0 \Rightarrow \text{data}$ ne désigne aucune zone allouée

$\text{capacity} > 0 \Rightarrow \text{data}$ désigne une zone pouvant contenir `capacity` entiers.

Une règle qui doit toujours rester vraie s'appelle un **invariant**.

Répondre à partir du code réel :

1. Quelles fonctions lisent le tableau ? Lesquelles le modifient ?
2. Quelles valeurs indiquent un succès ou un échec ?
3. Qui doit appeler `destroy_array` ?
4. Que se passe-t-il en cas d'oubli ? Et en cas de double libération ?
5. Comment la capacité évolue-t-elle lorsque le tableau est plein ?

Compiler d'abord en C :

```
gcc -std=c17 -Wall -Wextra -pedantic \
    int_array_c.c main_c.c -o int_array_c
```

Puis tenter de compiler `int_array_c.c` comme du C++ avec `g++`. Repérer les diagnostics liés à `malloc` et `realloc`. Ne pas ajouter de conversion uniquement pour faire disparaître l'erreur.

À retenir. Un programme C valide n'est pas forcément un bon programme C++.

Étape 1 — Faire fonctionner une première version C++

Comparer ensuite votre proposition avec `int_array.hpp`, `int_array.cpp` et `main.cpp`.

Commencer par une structure dont tous les champs ont une valeur connue :

```
struct IntArray {
    int capacity{};
    int size{};
    int* data{nullptr};
};
```

Utiliser `new[]` et `delete[]` pour le tableau. Lorsque le tableau est plein, conserver la même stratégie que la version C : la capacité est doublée. Procéder dans cet ordre :

1. créer un tableau deux fois plus grand ;
2. copier les `size` valeurs utiles ;
3. supprimer l'ancien tableau ;
4. remplacer `data`, puis mettre à jour `capacity`.

Adapter les paramètres en reprenant le vocabulaire du TD1 :

T	valeur	la fonction reçoit sa propre valeur
T&	modifier	la fonction peut modifier l'objet fourni
const T&	lire	la fonction lit sans modifier

Compiler séparément :

```
g++ -std=c++20 -Wall -Wextra -pedantic -c int_array.cpp
g++ -std=c++20 -Wall -Wextra -pedantic -c main.cpp
g++ int_array.o main.o -o int_array
```

L'option `-c` fabrique un fichier objet sans produire encore le programme final. La dernière commande assemble les deux fichiers objets.

À retenir. Pour agrandir : créer plus grand, copier, supprimer l'ancien, remplacer.

Étape 2 — Lire ou modifier ?

Ajouter ou vérifier les trois fonctions suivantes :

```
int at(const IntArray& array, int index);
int sum(const IntArray& array);
void fill(IntArray& array, int value);
```

Pour chacune, demander : « cette fonction doit-elle modifier le tableau ? »

Un indice est valide si `0 <= index < size`. Le dernier indice valide est donc `size - 1`. Signaler un indice invalide avec :

```
throw std::out_of_range{"IntArray index out of range"};
```

Prévoir `#include <stdexcept>`. Ici, `throw` sert simplement à signaler que l'indice demandé n'existe pas ; les exceptions seront étudiées plus tard.

Faire temporairement l'essai suivant dans `sum` :

```
array.size = 0; // doit etre refuse
```

Expliquer le diagnostic, puis supprimer cette ligne. Tester enfin `fill` sur plusieurs valeurs et vérifier que `size` et `capacity` ne changent pas.

À retenir. `const` indique ici qu'une fonction lit sans modifier.

Étape 3 — Deux façons d'utiliser `push_back`

Ajouter :

```
void push_back(IntArray& array, int value);
void push_back(IntArray& array, int value, int repetitions);
```

La première ajoute une fois la valeur ; la seconde l'ajoute `repetitions` fois. Plusieurs fonctions portant le même nom avec des paramètres différents forment une **surcharge**.

La seconde version doit appeler la première dans une boucle. Pour un nombre négatif, lancer `std::invalid_argument`. Tester :

```
push_back(values, 7);
push_back(values, 7, 3);
```

À retenir. Réutiliser une fonction existante évite de dupliquer le code.

Étape 4 — Présenter les opérations comme appartenant au tableau

Comparer les appels :

Avant	Après
<code>push_back(values, 7)</code>	<code>values.push_back(7)</code>
<code>at(values, 2)</code>	<code>values.at(2)</code>
<code>sum(values)</code>	<code>values.sum()</code>
<code>fill(values, 5)</code>	<code>values.fill(5)</code>

Une fonction placée dans la structure et appelée sur un objet est une **fonction membre**. Écrire :

```
struct IntArray {
    int capacity{};
    int size{};
    int* data{nullptr};

    void push_back(int value);
    void push_back(int value, int repetitions);
    int at(int index) const;
    int sum() const;
    void fill(int value);
};
```

Dans le fichier `.cpp`, une définition commence par exemple ainsi :

```
void IntArray::push_back(int value)
{
    // size et data sont les champs de l'objet utilise dans l'appel
}
```

`IntArray::push_back` signifie « le `push_back` qui appartient à `IntArray` ». Dans une fonction membre, écrire directement `size` ou `data` : la fonction sait déjà sur quel objet elle travaille.

Faire le parallèle :

```
int sum(const IntArray& array); // fonction libre
int IntArray::sum() const;      // fonction membre
```

Dans la seconde écriture, le `const` placé après les paramètres signifie que `sum` ne modifie pas l'objet sur lequel elle est appelée. Vérifier que `size = 0;` est refusé dans `sum() const`.

À retenir. `values.sum()` présente `sum` comme une opération de `values`. Le `const` final signifie que cette opération ne modifie pas `values`.

Étape 5 — Vérifier que le tableau reste correct

Écrire des tests pour les situations suivantes :

1. quelques insertions sans agrandissement ;
2. suffisamment d'insertions pour provoquer plusieurs agrandissements ;
3. première et dernière valeurs ;
4. indices invalides `-1` et `size` ;
5. somme ;
6. remplissage ;
7. destruction.

Lors de chaque agrandissement, vérifier que les valeurs déjà insérées sont toujours présentes.

Compiler également avec :

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined -g \
    int_array.cpp main.cpp -o int_array_san
```

Ces options ajoutent des vérifications capables de repérer de nombreuses erreurs mémoire pendant l'exécution. L'absence de diagnostic signifie seulement que les tests exécutés n'ont pas révélé ces erreurs.

À retenir. Tester un tableau, c'est aussi tester les changements de capacité et les limites des indices.

Étape 6 — Les deux problèmes qui restent

Le code extérieur peut encore écrire :

```
values.size = -1;
values.capacity = 0;
values.data = nullptr;
```

Pourquoi le compilateur l'accepte-t-il ? Quelles règles ces lignes peuvent-elles casser ? Le mot **encapsulation** désigne l'idée d'empêcher l'utilisateur de modifier directement les données internes qui doivent rester cohérentes.

Répondre ensuite avec des mots simples :

1. Comment créer directement un `IntArray` correct ? En C++, l'opération spéciale de création s'appelle un **constructeur**.
2. Comment libérer automatiquement la mémoire quand l'objet disparaît ? La fonction spéciale appelée à ce moment s'appelle un **destructeur**.
3. Que se passe-t-il avec `IntArray b = a;` ? Les deux champs `data` contiennent-ils la même adresse ?

Phrase essentielle : copier un pointeur ne copie pas ce qu'il pointe. Ne pas implémenter aujourd'hui la copie profonde ni la classe finale.

Proposer seulement l'utilisation souhaitée pour la suite :

```
IntArray values{4};
values.push_back(7);
values.push_back(10);
std::cout << values.sum() << '\n';

// values.size = -1; // ne devra plus etre permis
```

En C++, on cherche à faire en sorte que l'objet s'occupe lui-même de sa mémoire pendant toute sa durée de vie. Cette idée conduit au RAII ; il n'est pas demandé d'en apprendre l'acronyme dans ce TD.

À retenir. Les fonctions membres ne suffisent pas : les champs restent directement modifiables et la mémoire est encore gérée à la main.

À rendre. La version à fonctions libres, la version à fonctions membres, les tests, et une page maximum répondant aux questions de l'étape 6.